



PROCESADORES DE LENGUAJE DRLL

Grupo 206

11/03/2025

Jorge Mejías Donoso 100495807@alumnos.uc3m.es

Alberto Menchen Montero 100495692@alumnos.uc3m.es

ÍNDICE

1. rd_lex():	3
1.1 Qué es:	3
1.2 Elementos básicos proporcionados:	3
2. Diseño de la gramática inicial:	3
2.1 Definición de la Gramática	3
2.3 Descripción de la Gramática	4
2.4 Explicación de la Gramática	4
3. Transformación de la gramática a LL(1):	5
3.1 Definición de la Gramática LL(1)	5
3.2 Gramática LL(1)	5
3.3 Descripción de la Gramática LL(1)	6
4. Distinción entre nivel lexico y sintactico	6
4.1 Nivel Léxico (Manejado por rd_lex())	6
4.2 Nivel Sintáctico (Manejado por el Parser Descendente Recursivo)	7
5. Resumen del diseño del parser.	7
5.1 Estrategia de Implementación	8
5.2 Funciones Principales	8
6. Diagrama sintáctico representando la gramática diseñada.	10
7. Pruebas de Traducción y Evaluación Realizadas	12
7.1. Pruebas Correctas	12
7.2. Casos Límite	12
7.3. Casos Incorrectos (Errores Sintácticos)	13
7.4. Conclusión	14

1. rd_lex():

1.1 Qué es:

- Es el analizador léxico del programa. Su función es leer caracteres de la entrada, identificarlos y clasificarlos en tokens para que el parser pueda procesarlos correctamente.

1.2 Elementos básicos proporcionados:

- **Números** (T_NUMBER): Se detectan como secuencias de uno o más dígitos consecutivos. Se almacenan en tokens.number y se devuelve el token T_NUMBER.
- **Variables** (T_VARIABLE): Se reconocen como:
 - Una sola letra (A, x, y).
 - Una letra seguida de un único dígito (A1, x2).
 - Se almacenan en tokens.variable_name y se devuelve el token T_VARIABLE.
- **Operadores aritméticos** (T_OPERATOR): Se identifican los operadores +, -, *, /. Se almacenan en tokens.token_val y se devuelve el token T_OPERATOR.
- **Caracteres especiales y literales**: Cualquier otro carácter que no sea un número, variable u operador se devuelve directamente como su código ASCII. Esto incluye:
 - Paréntesis → (,)
 - Signo de igualdad → =
 - Salto de línea → \n
- **Fin de archivo (EOF)**: Si se encuentra el final del archivo, el programa finaliza.

2. Diseño de la gramática inicial:

2.1 Definición de la Gramática

Para representar la sintaxis de las expresiones aritméticas en notación prefija, hemos diseñado la siguiente gramática. Esta define cómo se estructuran las expresiones y establece las reglas de producción necesarias para su correcta interpretación.

2.2 Gramática

Unset

```
<Programa> ::= <Expresion> "\n"
<Expresion> ::= <Numero> | <Variable>
               | "(" <Operador> <Expresion> <Expresion> ")"
               | "(" "=" <Variable> <Expresion> ")"
<Operador> ::= "+" | "-" | "*" | "/"
```

```
<Numero> ::= <Digito> | <Numero> <Digito>
<Variable> ::= <Letra> | <Letra> <Digito>
<Digito> ::= "0" | "1" | "2" | ... | "7" | "8" | "9"
<Letra> ::= "A" | "B" | "C" | ... | "Z" |
           | "a" | "b" | "c" | ... | "z"
```

2.3 Descripción de la Gramática

- Estructura general:
 - <Programa> → Define una única expresión seguida de un salto de línea (\n).
 - <Expresion> → Puede ser:
 - Un número (<Numero>).
 - Una variable (<Variable>).
 - Una operación aritmética en notación prefija:
 - (<Operador> <Expresion> <Expresion>)
 - Una asignación en notación prefija:
 - (= <Variable> <Expresion>)
- Definición de componentes:
 - <Operador> → +, -, *, /.
 - <Numero> → Número formado por uno o más dígitos.
 - <Variable> → Letras (A, x, y) o letras seguidas de un dígito (A1, x2).
 - <Digito> → 0-9.
 - <Letra> → A-Z, a-z.

2.4 Explicación de la Gramática

La regla principal es <Programa>, que indica que una entrada válida consiste en una única <Expresion> seguida de un salto de línea (\n). Esto garantiza que cada línea de entrada contenga una expresión completa y evita fragmentos incompletos o errores de formato.

El elemento central de la gramática es <Expresion>, que define cuatro tipos de expresiones:

1. **Números (<Numero>):** Representan valores enteros y pueden estar formados por uno o más dígitos.
2. **Variables (<Variable>):** Son identificadores que pueden ser una sola letra (A, x, y) o una letra seguida de un dígito (A1, x2).
3. **Operaciones aritméticas en notación prefija:** Se expresan como (<Operador> <Expresion> <Expresion>). Aquí, el operador

(+, -, *, /) aparece antes de los dos operandos. Por ejemplo, la expresión prefija (+ A B) equivale a $A + B$ en notación infija.

4. **Asignaciones:** Se representan como (`<"=">` `<Variable>` `<Expresion>`), permitiendo asignar el resultado de una expresión a una variable. Por ejemplo, `(= x (+ 1 2))` indica que `x` toma el valor $1 + 2$.

Los operadores definidos en `<Operador>` incluyen los básicos de la aritmética (+, -, *, /). Esta gramática solo permite operadores binarios, lo que significa que cada operador debe aplicarse a exactamente dos operandos. No se incluyen operadores unarios, por lo que expresiones como `(- 5)` no son válidas.

Las reglas `<Numero>` y `<Variable>` establecen cómo se construyen los operandos. `<Numero>` se define de manera recursiva (`<Digito>` | `<Numero>` `<Digito>`), lo que permite representar números de múltiples cifras. Por otro lado, `<Variable>` admite nombres de una sola letra o combinaciones de letra y dígito, asegurando simplicidad y facilidad de reconocimiento en el parser.

Finalmente, `<Digito>` y `<Letra>` garantizan que los números y variables sólo contengan caracteres válidos. `<Digito>` está limitado a los valores del 0 al 9, mientras que `<Letra>` admite letras en mayúsculas y minúsculas del alfabeto.

3. Transformación de la gramática a LL(1):

3.1 Definición de la Gramática LL(1)

Para garantizar que la gramática sea adecuada para un parser descendente recursivo, realizamos modificaciones que eliminan ambigüedades y recursión por la izquierda, obteniendo una versión en forma LL(1).

3.2 Gramática LL(1)

Unset

```
<Programa> ::= <Expresion> "\n"
<Expresion> ::= <Numero>
               | <Variable>
               | "(" <InicioExpresion> ")"
<InicioExpresion> ::= <Operador> <Expresion> <Expresion>
                    | "=" <Variable> <Expresion>
<Operador> ::= "+" | "-" | "*" | "/"
<Numero> ::= <Digito>
<Variable> ::= <Letra>
```

```
<Digito> ::= "0" | "1" | "2" | ... | "7" | "8" | "9"  
<Letra>  ::= "A" | "B" | "C" | ... | "Z" | "a" | "b"  
          | "c" | ... | "z"
```

3.3 Descripción de la Gramática LL(1)

- Estructura general:
 - <Programa>: Define una única expresión seguida de un salto de línea (\n).
 - <Expresion>: Puede ser un número (<Numero>), una variable (<Variable>), o una estructura prefija con paréntesis ((<InicioExpresion>)).
 -
 - <InicioExpresion>: Diferencia entre operaciones aritméticas (<Operador> <Expresion> <Expresion>) y asignaciones ("=" <Variable> <Expresion>).
- Definición de componentes:
 - <Operador>: Incluye +, -, *, /.
 -
 - <Numero>: Se define directamente como un <Digito>, eliminando recursión innecesaria.
 -
 - <Variable>: Se limita a una única <Letra>, permitiendo que el análisis léxico maneje nombres más complejos.
 -
 - <Digito> y <Letra>: Definen los caracteres válidos para números y variables.

4. Distinción entre nivel lexico y sintactico

Para entender la implementación del analizador, es importante diferenciar entre los aspectos léxicos y sintácticos de la gramática.

4.1 Nivel Léxico (Manejado por rd_lex())

El analizador léxico (rd_lex()) se encarga de identificar y clasificar los elementos básicos de la entrada antes de que el análisis sintáctico los procese. Los componentes léxicos definidos en la gramática LL(1) incluyen:

- <Digito>: Se reconoce cualquier número del 0 al 9 y se agrupa en un token T_NUMBER.
- <Letra>: Identifica caracteres alfabéticos (A-Z, a-z) y los categoriza como parte de un T_VARIABLE.
- Operadores (+, -, *, /): Se clasifican como T_OPERATOR.
- Paréntesis ((,)) y el signo de asignación (=) son devueltos como tokens literales.
- Saltos de línea (\n): Indican el final de una expresión.

El analizador léxico no interpreta la estructura de la expresión, simplemente convierte los caracteres en tokens.

4.2 Nivel Sintáctico (Manejado por el Parser Descendente Recursivo)

El análisis sintáctico organiza los tokens generados por el análisis léxico siguiendo las reglas de la gramática LL(1).

- <Programa>: Asegura que la entrada sea una única <Expresion> seguida de un (\n)
- <Expresion>: Diferencia entre números, variables y estructuras prefijas con paréntesis.
- <InicioExpresion>: Diferencia entre operaciones aritméticas (<Operador> <Expresion> <Expresion>) y asignaciones ("=" <Variable> <Expresion>).

El parser detecta si los tokens siguen el orden correcto según la gramática LL(1). Si la secuencia de tokens no coincide con las reglas definidas, se genera un error sintáctico.

Con esta separación clara, la implementación divide el procesamiento en dos fases: primero el análisis léxico, que tokeniza la entrada, y luego el análisis sintáctico, que valida y estructura la expresión.

Se realizaron ajustes clave para convertir la gramática en LL(1):

- Se eliminó la ambigüedad en <Expresion>, separando operadores y asignaciones en <InicioExpresion>.
- Se evitó la recursión innecesaria en <Numero> y <Variable>, delegando la interpretación de valores más complejos al analizador léxico.
- La gramática resultante permite analizar expresiones sin necesidad de retroceso, facilitando la implementación de un parser descendente recursivo.

Con esta transformación, la gramática se vuelve predecible y adecuada para su implementación, permitiendo una correcta traducción de notación prefija a infija.

5. Resumen del diseño del parser.

El diseño del parser se basa en la implementación de un **parser descendente recursivo** que procesa expresiones en notación prefija y las convierte en notación infija. La traducción se realiza de manera recursiva siguiendo las producciones de la gramática LL(1) definida previamente.

5.1 Estrategia de Implementación

Para la implementación del parser, se han definido varias funciones en el archivo `drLL.c`, cada una correspondiente a una producción de la gramática. La estructura del parser sigue el modelo clásico de un **analizador sintáctico recursivo**, donde cada no terminal de la gramática tiene una función dedicada en el código.

El parser recibe tokens generados por la función `rd_lex()`, que implementa el **análisis léxico**, y los analiza según las reglas de la gramática definida.

5.2 Funciones Principales

A continuación, se detallan las funciones principales implementadas y su relación con la gramática, incluyendo fragmentos de código relevantes que ayuden a entender la implementación:

`ParseAxiom()`

Esta función representa la producción inicial del parser y garantiza que se procesen expresiones completas seguidas de un salto de línea (`\n`). Llama a la función principal `PaerseYourGrammar()`, asegurando que se interprete correctamente la expresión prefija proporcionada.

```
C/C++
void ParseAxiom () {
    PaerseYourGrammar ();
    printf("\n");
    if (tokens.token == '\n') {
        MatchSymbol('\n');
        printf("\n");
    } else {
        rd_syntax_error(-1, tokens.token, "-- Unexpected Token
(Expected:%d=None, Read:%d) at end of Parsing\n");
    }
}
```

`PaerseYourGrammar()`

Esta función actúa como punto de entrada del parser y llama a `ParseExpresion()` para procesar la expresión principal de acuerdo con la gramática definida.

C/C++

```
void PaerseYourGrammar() {  
    ParseExpresion();  
}
```

ParseExpresion()

Se encarga de procesar una **expresión aritmética o una asignación**. Distingue entre:

- **Números (T_NUMBER)**: Se imprimen directamente.
- **Variables (T_VARIABLE)**: Se imprimen directamente.
- **Expresiones complejas**: Se delegan a `ParseInicioExpresion()` si están entre paréntesis.

C/C++

```
void ParseExpresion() {  
    if (tokens.token == T_NUMBER) {  
        printf("%d", tokens.number);  
        MatchSymbol(T_NUMBER);  
    } else if (tokens.token == T_VARIABLE) {  
        printf("%s", tokens.variable_name);  
        MatchSymbol(T_VARIABLE);  
    } else if (tokens.token == '(') {  
        MatchSymbol('(');  
        ParseInicioExpresion();  
        MatchSymbol(')');  
    } else {  
        rd_syntax_error(-1, tokens.token, "Se esperaba una  
expresión, pero se encontró %d\n");  
    }  
}
```

ParseInicioExpresion()

Implementa `<InicioExpresion>`, procesando los casos:

- **Expresión aritmética**: Si el primer token es un operador (+, -, *, /), procesa los dos operandos de manera recursiva.
- **Asignación**: Si el primer token es =, procesa la variable y la expresión asignada.

```
C/C++
void ParseInicioExpresion() {
    if (tokens.token == T_OPERATOR) {
        int op = tokens.token_val;
        MatchSymbol(T_OPERATOR);
        printf("(");
        ParseExpresion();
        printf(" %c ", op);
        ParseExpresion();
        printf(")");
    } else if (tokens.token == '=') {
        MatchSymbol('=');
        printf("(");
        printf("%s = ", tokens.variable_name);
        MatchSymbol(T_VARIABLE);
        ParseExpresion();
        printf(")");
    } else {
        rd_syntax_error(-1, tokens.token, "Se esperaba un
operador o '=', pero se encontró %d\n");
    }
}
```

Esta función es fundamental para convertir la expresión prefija en notación infija, ya que determina el orden correcto de las operaciones y asignaciones.

El parser descendente recursivo implementado es capaz de analizar expresiones en notación prefija y traducirlas correctamente a notación infija. La implementación sigue fielmente la gramática LL(1) diseñada y respeta las reglas de precedencia y asociatividad de los operadores. La división modular en funciones permite una estructura clara y facilita la extensión del código si fuera necesario.

6. Diagrama sintáctico representando la gramática diseñada.

Una vez que entendemos la distinción entre estos dos niveles podemos pasar a crear la gramática en JFLAP para comprobar que realmente es LL(1). En este programa solo introducimos la parte que corresponde al nivel sintáctico. Donde nos queda la siguiente gramática.

LHS		RHS
P	→	En
E	→	d
E	→	v
E	→	(I)
I	→	OEE
I	→	=vE
O	→	+
O	→	-
O	→	*
O	→	/

Donde d y v hacen referencia a los dígitos y las variables pero por resumir las pruebas se meten esas dos letras. Probamos un caso como por ejemplo $(=v(+dd))n$ el cual traducido significa lo siguiente $(v = (d + d)) \setminus n$. Vemos cómo se genera el siguiente árbol de derivación y en la pila no queda nada más que el símbolo \$ y por lo tanto ha terminado con éxito la ejecución

Start Step Noninverted Tree

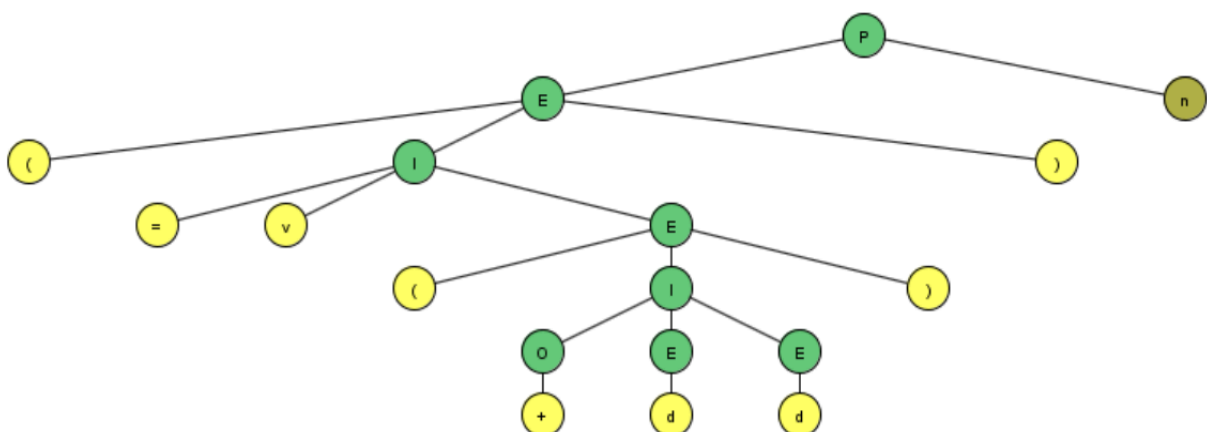
Input

(=v(+dd))n

Input Remaining

\$

Stack



7. Pruebas de Traducción y Evaluación Realizadas

Se presentan las pruebas diseñadas para evaluar el correcto funcionamiento del parser descendente recursivo. Estas pruebas incluyen **casos correctos**, **casos límite** y **casos incorrectos** que deben ser rechazados. Se muestra la entrada y la salida esperada generada por el programa.

7.1. Pruebas Correctas

Expresiones aritméticas simples

- **Entrada:** $(+ 3 4)$
- **Salida esperada:** $(3 + 4)$
- **Entrada:** $(* 2 (+ 1 3))$
- **Salida esperada:** $(2 * (1 + 3))$

Expresiones con variables

- **Entrada:** $(+ A B)$
- **Salida esperada:** $(A + B)$
- **Entrada:** $(* (- X 2) Y)$
- **Salida esperada:** $((X - 2) * Y)$

Expresiones con asignaciones

- **Entrada:** $(= A (+ 1 2))$
- **Salida esperada:** $(A = (1 + 2))$
- **Entrada:** $(= X (* Y Z))$
- **Salida esperada:** $(X = (Y * Z))$
- **Entrada:** $(= A (= B (= C 10)))$
- **Salida esperada:** $(A = (B = (C = 10)))$

Expresiones anidadas

- **Entrada:** $(+ (* 2 3) (- 5 1))$
- **Salida esperada:** $((2 * 3) + (5 - 1))$
- **Entrada:** $(= A (+ (* B C) (- D 4)))$
- **Salida esperada:** $(A = ((B * C) + (D - 4)))$

7.2. Casos Límite

Expresiones con un solo número o variable

- **Entrada:** 5
- **Salida esperada:** 5

- **Entrada:** X
- **Salida esperada:** X

Expresiones con operaciones sin operandos

- **Entrada:** (+)
- **Salida esperada:** ERROR in line 1 Se esperaba una expresión, pero se encontró -1
- **Entrada:** (= X)
- **Salida esperada:** ERROR in line 1 Se esperaba una expresión, pero se encontró -1

Expresiones con anidamientos excesivos

- **Entrada:** (+ (+ (+ 1 2) 3) 4)
 - **Salida esperada:** (((1 + 2) + 3) + 4)
-

7.3. Casos Incorrectos (Errores Sintácticos)

Operadores sin suficientes operandos

- **Entrada:** (+ 1)
- **Salida esperada:** ERROR in line 1 Se esperaba una expresión, pero se encontró -1

Expresiones mal formateadas

- **Entrada:** (* 2 3
- **Salida esperada:** ERROR in line 2 token 41 expected, but 10 was read
- **Entrada:** * 2 3)
- **Salida esperada:** ERROR in line 1 Se esperaba una expresión, pero se encontró -1

Uso incorrecto de variables y números

- **Entrada:** (+ A 3B)
 - **Salida esperada:** ERROR in line 1 token 41 expected, but 1003 was read
 - **Entrada:** (= 2 A)
 - **Salida esperada:** ERROR in line 1 token 1003 expected, but 1001 was read
-

7.4. Conclusión

Estas pruebas cubren una amplia gama de casos, asegurando que el parser funciona correctamente en situaciones esperadas y rechaza las expresiones sintácticamente incorrectas. La implementación ha sido verificada para cumplir con las especificaciones de la gramática LL(1) y los requisitos de la práctica.